

REPORT ON OO CODE METRICs

Software Metrics -PA2559(GROUP 11)

1st AHMED MAHFOOZ ALI KHAN 2nd TAFAZZUL HAZQUEEL SYED 3rd SATISH RAJ SOMAROUTHU

Dept of Computer Science
20030116-8555
alaa23@student.bth.se

Dept of Comptuer Science
20030313-1437
tasy23@student.bth.se

Dept of Comptuer Science
20020727-2915
saso23@student.bth.se

4th KOSARI SUHAS
Dept of Comptuer Science
20010611-0075
suko23@student.bth.se

Abstract—This paper looks at how software metrics give a diverse review on certain measurements, called object-oriented (OO) metrics, to understand how they work, impact and improve software development. We looked at ten different research papers to see which metrics are commonly used and how they're helpful. By studying these metrics and how they're used in actual software projects, we gained more knowledge about them. This research can be valuable for software developers or software managers because it helps them pick the best metrics for their projects and make sure that it makes sense as well as maturity to compare between the metrics.

Index Terms—Object-oriented (OO) metrics, goals, attributes, metrics maturity assessing criteria, open source systems(object-oriented)

I. INTRODUCTION

Software metrics involve quantifiable measures of various characteristics within software systems. The aim of identifying and analyzing these metrics is to evaluate ongoing projects, products, or processes and enhance the overall quality of the software system upon completion. This process provides valuable insights into performance, reliability, and maintainability.

Given the widespread adoption of the Object-Oriented (OO) paradigm, this review concentrates on OO design metrics, essential for assessing and improving the quality of OO applications. These metrics not only highlight potential design flaws or anti-patterns that can affect code quality but also identify performance bottlenecks that impact scalability and responsiveness. OO metrics are instrumental in assessing code complexity, as well as evaluating reliability (such as error handling, fault tolerance, and defensive programming) and maintainability (including readability, ease of modification, and re-usability).

Therefore, the purpose of this review is to examine selected research papers focusing on: (a) Identification and categorization of OO metrics, (b) Categorizing the goals of employed OO metrics, (c) Classifying attributes of the code based on these metrics, (d) Proposing maturity criteria to assess these metrics, (e) Applying the proposed criteria to evaluate OO metrics, and (f) Categorizing the systems studied in the referenced papers.

Additionally, this review will discuss the use of tools for extracting OO metrics to analyze software systems effectively.

II. OBJECT-ORIENTED METRICS

A. Description

Object-oriented metrics are quantitative measurements that are employed to assess the characteristics of OO software applications. Numerous software metrics have been devised for OO paradigms, encompassing aspects such as classes, objects, abstraction, inheritance and so on to calculate various attributes like software quality, cohesion, size, complexity, maintainability, coupling etc. Table 1 presents a comprehensive list of the metrics identified across various research papers.

B. Characterisation

Process, product and project metrics are three primary categories of metrics used in software development realm. In similar fashion, Object-oriented metrics can be classified into two major categories.

a) **Static Metrics:** Size Metrics, Complexity metrics, Cohesion metrics, Coupling metrics, etc.

b) **Dynamic Metrics:** Test coverage metrics, Execution metrics, Memory Metrics, etc.

However, given that the research papers focus specifically on static metrics that assess code quality, the following categories of metrics will be considered in this context:

- **Size Metrics:** These metrics quantify the size of software components, typically in terms of lines of code (LOC), number of classes, methods, or other structural elements.
- **Complexity Metrics:** These metrics evaluate the complexity of the codebase, measuring factors such as cyclomatic complexity, nesting depth, or number of dependencies.
- **Cohesion Metrics:** These metrics gauge how well the elements within a module or class are related to each other. High cohesion indicates that elements within a module are closely related and focused on a single task.

TABLE I: Description of OO metrics

METRICS	DESCRIPTION	PAPER(s)
LOC	Code lines for each class (all non-empty, non-comment lines in the class's methods)	[1,3]
CLOC	Number of lines with comments (lines with comments in each class)	[1]
NOF	Number of functions (functions per class)	[1]
NCLASS	Number of classes	[1]
STAT	Number of statements in a class	[1]
NINT	Num of interfaces	[1]
NSTR	No of structures	[1]
NOS	Avg Num of statements per method	[1]
NOM	Avg Num of methods per class	[1]
ABD	Avg block depth	[1]
CCMax	Maximum cyclomatic complexity of a class's method	[1,3,4]
Avg CC	AVG cyclomatic complexity of non-abstract methods in each class	[1,4]
Sqale_index	Effort to fix all code smell issues (in minutes)	[2]
Reliability_remediation_effort	Effort to fix all bug issues (in minutes)	[2]
Security_remediation_effort	Effort to fix all vulnerability issues (in minutes)	[2]
Total_principal	Effort to fix all issues (sum of code smell, bug, and vulnerability remediation effort)	[2]
Code_smells	Total number of code smell issues of a project	[2]
Uncovered_lines	Number of lines of code not covered by unit tests	[2]
Duplicated_blocks	Number of duplicated blocks of lines of a project	[2]
SDMC	SD method complexity (Standard deviation of method complexity in a class)	[3,4]
WMC	Weighted methods per class (Sum of complexities of local methods of a class)	[2,3,4,6]
NIM	Number of instance methods (methods in an instance object of a class)	[3,4]
NTM	Number of trivial methods (local methods in the class)	[3,4]
NLM	Number of local methods (methods implemented in a class)	[3]
AMC	Average method complexity (average cyclomatic complexity of all methods in a class)	[2,3]
NCM	Number of class methods (class methods implemented in a class)	[3]
ALOC	Avg lines of code (average executable lines of code of a class)	[3,4]
Modified	Modified cyclomatic complexity (excluding each case statement)	[5]
Essential	Essential cyclomatic complexity (code structuredness by counting cyclomatic complexity)	[5]
Strict	Strict cyclomatic Complexity (identical to cyclomatic complexity, excluding AND and OR operators)	[5]
HK	Henry Kafura (measures information flow relative to function size)	[5]
Nesting	Nesting complexity (maximum nesting level of control constructs in a function)	[5]
FANOUT	Number of outputs (updates to parameters and global variables)	[5,8]
CountPath	CountPath complexity (number of unique decision paths through code)	[5]
WMP	Weighted methods per class (count of local methods defined within a class)	[5]
SLOC	Source lines of code (executable lines of source code)	[5,8]
CC McCabe	McCabe's Cyclomatic complexity (number of independent paths through a program unit)	[5]
FANIN	Number of inputs (parameters, global variables, and method calls)	[5,8]
CBO	Coupling between objects (number of distinct non-inheritance-related classes a class depends on)	[2,5,6]
LCOM	Lack of cohesion of methods (number of pairs of methods without common attributes)	[2,5,6]
RFC	Response for a class (total local methods including inherited methods)	[2,5]
CC	Complexity based on the number of decisions in source code	[6,8]
NOCC	Number of Class Children	[6]
MSP	Message-passing coupling (number of distinct methods called)	[6]
LCOMN	Lack of Cohesion Methods Negative	[7]
NPATH	Number of possible execution paths in a method	[8]
EVG	Essential complexity (modified cyclomatic complexity)	[8]
NOQ	Number of Queries per class	[9]
Dcy	Number of Dependencies	[9]
CONS	Number of Constructors	[9]
NAA	Number of Added Attributes	[9]
PDcy	Number of Package Dependencies	[9]
NOAC	Number of Operations Added	[9]
JLOC	JavaDoc Lines of Code	[9]
NOOC	Number of Operations Overridden	[9]
DCI	Data, Context, Interface	[10]
DCD	Design Complexity Distance	[10]
TCC	Tight Class Cohesion	[10]
CAMC	Ratio of total number of 1s in the matrix to the total size of the matrix	[10]
SNHD	Scaled NHD (closeness of NHD metric to the maximum value)	[10]
SCC	Similarity-based Class Cohesion (weighted average of method-method, attribute-attribute metrics)	[10]

TABLE II: Categorisation of the OO Metrics

S.NO	CATEGORY	METRICS
1	Size	WMC, LOC, SLOC, AvgLOC, NIM, NCM, NLM, NPM, DAM
2	Complexity	SDMC, AMC, CCMAX, CC, EVG, NPATh, FANIN, FANOUT, WMC McCabe
3	Coupling	MPC, RFC, CBO, Ca Afferent coupling, Ce Efferent coupling, CBM, IC
4	Cohesion	LCOM, LCOMN, CAM, MOA, TCC, DCD, DCI, CAMC, NHD, SNHD, D3C2, SCC
5	Inheritance	DIT, NOC, CBC, NOCC

- **Coupling Metrics:** These Metrics assess interdependence between modules or classes. Low coupling suggests greater independence and minimal impact of changes between modules.
- **Coupling Metrics:** These Metrics assess interdependence between modules or classes. Low coupling suggests greater independence and minimal impact of changes between modules.

TABLE II shows the categorisation of the OO metrics identified in the given research papers.

III. THE GOALS

A. Description

Measures that are goal-based concentrate on the improvements that must be made in a specific situation. As soon as it has decided what needs to be measured—identifying and categorising the entities to be investigated—and what process needs to be measured—identifying and allocating pertinent metrics—it will always set targets. TABLE IV will outline the goals or explanation behind the use of OO metrics in the research publications.

B. Categorisation of Goals

Object-oriented metrics offer a quantitative assessment of various aspects of software design, such as cohesion, coupling, and complexity. This allows developers to objectively measure the health of their codebase and identify potential design flaws or areas for improvement early in the development process.

TABLE III: Categorisation of Goals

Category	Research Paper(s)
Changeability	[1]
Reliability	[4]
Security	[5]
Fault-Prediction	[3,7]
Maintainability	[2,6,9]
Cohesion	[10]
Coupling	[8]

TABLE IV: Identification and Goal Description

Paper	Goal Description
[1]	This study aims to evaluate the comparison of object-oriented (OO) metrics versus organizational metrics in forecasting code churn. also the most effective OO metric for this purpose.
[2]	This research aims to identify key software metrics as accurate predictors of technical debt, and also studies the efficiency of ML techniques in forecasting technical debt
[3]	This study aims to refine a previous approach using odds ratio to evaluate how complexity metrics predict faults in classes of OO software systems. It includes a comparative analysis of widely used and lesser-known metrics using data from multiple versions of Eclipse.
[4]	This study aims to evaluate the effectiveness of OO metrics like Lines of Code (LOC), McCabe's cyclomatic complexity (MCC), and Response for Class (RFC), in identifying classes within agile or iterative software development
[5]	This research aims to investigate if metrics: complexity, coupling, and cohesion, can be used to predict software vulnerabilities and their relationship with these metrics. Also to evaluate the performance of these metrics, validated through a case study of Mozilla Firefox.
[6]	This study aims to refine the selection process for refactoring candidates (high coupling and size) and propose a metrics-based approach by considering class characteristics and a benchmark of real-world data
[7]	This research aims to evaluate the effectiveness of fault prediction with the help of OO metrics (like WMC, DIT, and RFC) in an open-source system.
[8]	The goal states that It challenges summation's dominance and can significantly impact the correlations among metrics and between metrics, a defect counts. Finding it often suboptimal compared to alternatives like mean or median aggregation. Diversifying aggregation techniques could improve defect prediction model performance, particularly in identifying defect-prone files.
[9]	This study diversifies the aggregation techniques challenges summation's dominance, impacting metric correlations and defect counts. Mean or median aggregation often outperforms expectations, especially in identifying defect-prone files.
[10]	this study aims to improve and enhance OO class cohesion metrics and study the effectiveness of these metrics in fault prediction and maintainability software systems

IV. CODE ATTRIBUTES

A. Description

In object-oriented programming, code attributes refers to distinct characteristics or properties of the codebase that can be assessed and analysed to evaluate the software's quality, complexity, or maintainability. Various metrics can be employed to quantify these attributes, facilitating informed decisions for code optimization and enhancement. Size, Maintainability, Coupling, Inheritance, Cohesion, etc are some common examples. Table V illustrates the attributes identified within the research papers.

TABLE V: Identified Attributes

ATTRIBUTE	DESCRIPTION	PAPER(s)
Size	The size (population, quantity, dimensions, capabilities) of the code	[1,2,3,4,5,6,7,9,10]
Maintainability	How readily can software be modified or updated	[1,2,9,10]
Cohesion	The extent to which components within a software module are interconnected	[5,6,7,9,10]
Complexity	The complexity of the software code	[1,3,4,5,6,7,8,10]
Coupling	The level of interdependence among components within a software module	[2,5,6,7,8,9]
Inheritance	The reuse of code in OOP	[2,5,6,7,9]
Reliability	The effectiveness of software's performance/metric across varying conditions	[2,5,6,8,9]
Effectiveness	The ability of a software system to achieve its intended purpose or objectives	[1,2,4,7,10]
Changeability	The ability of a software system to accommodate changes or modifications easily and efficiently	[2]
Fault-Finding-Ability	The ability of a system or software to effectively identify and diagnose issues or faults	[3,5,7,8,10]
Algorithm Complexity	The analysis and quantification of computational resources required by an algorithm to solve a problem	[2,5]
No. of Faults Found	The quantity or count of defects, errors, or issues identified in a software system during testing or operation	[8]

V. PROPOSED CRITERIA

B. Categorisation of Attributes

In the context of categorizing attributes, we can consider two main categories: **Internal** attributes and **External** attributes. These categories help us understand whether the attribute is intrinsic to an entity or influenced by external factors.

- **Internal Attributes:** These attributes refer to characteristics or qualities that are inherent to an entity and are typically under its direct control. These attributes are often fundamental to understanding the nature or behavior of the entity itself.

Ex:Size,Cohesion,Coupling....etc

- **External Attributes:** These attributes refer to characteristics or properties of software components that can be observed or measured based on the external behavior of those components and focuses on how the software interacts with its environment, users, or other systems.

Ex:Reliability,Maintainability,Effectiveness...etc

TABLE VI: Categorisation Of Attributes

PAPER	ATTRIBUTES	
	INTERNAL	EXTERNAL
[1]	Size, Complexity	Maintainability, Effectiveness
[2]	Size,Coupling, Inheritance, Algorithm Complexity	Reliability,Maintainability, Effectiveness,Changeability
[3]	Size,Complexity	Fault-Finding-Ability
[4]	Size, Complexity	Effectiveness
[5]	Complexity,Cohesion, Inheritance,Coupling, Size	Reliability, Fault-Finding-Ability
[6]	Size,Cohesion, Inheritance,Coupling,	Reliability
[7]	Size,Coupling Inheritance,Complexity	Effectiveness, Fault-Finding-Ability
[8]	Size,Coupling Inheritance,Complexity	Reliability, Fault-Finding-Ability
[9]	Complexity,Size, Coupling	Maintainability
[10]	Cohesion	Reliability

A. Description

Proposed criteria for object-oriented metrics refer to specific measures or indicators used to evaluate the quality, complexity, and design attributes of object-oriented(OO)software systems. These metrics are valuable in assessing the maintainability,re-usability, and overall efficiency of object-oriented code.However, it's essential to interpret these metrics judiciously and in context, considering specific project requirements and design goals.**Table VII** contains the criteria which are proposed to assess the maturity of identified metrics including the size, complexity, cohesion, coupling, and inheritance..etc

B. Justification

In software engineering, metrics like size, complexity, cohesiveness, inheritance, and coupling are frequently employed to evaluate the calibre of programme design and execution. Accuracy, reliability, Predictive Power, consistency, Maintainability, are a few of the criteria that should be used to evaluate these measures, though. Some Justification for the suggested standards in TABLE VII that can be used to evaluate the maturity of the identified metrics are provided in TABLE VIII.

TABLE VII: Maturity assessing Criteria

CRITERIA	DESCRIPTION
Accuracy	ensures the metric genuinely measures the OO constructs it intends to involving construct, content, and criterion validity to gauge its theoretical comprehensive, and outcome-related accuracies.
Reliability	underscores the metric's consistency across various conditions, assessed through test-retest reliability for temporal stability, inter-rater reliability for observer consistency, and internal consistency for composite metrics' coherence
Predictive Power	The ability of the metric to predict future characteristics or outcomes, such as potential maintenance issues, based on current measurements.
Maintainability	The ease with which software can be updated, enhanced, or corrected. It encompasses the software's readability, testability, and simplicity, impacting the effort and cost associated with modifications.The software can adapt to changes efficiently, making it a key factor in software quality and longevity.
Consistency	It involves the metric's ability to provide coherent results across different levels of abstraction (e.g., class level, method level) and examines the metric's uniform applicability across different abstraction levels and OO languages, focusing on cross-level and cross-language consistency.

TABLE VIII: Justification on Criteria

CRITERIA	JUSTIFICATION
Accuracy	Ensures that the metric measures what it is intended to measure, making it foundational to the metric's usefulness. It's crucial that these metrics accurately reflect the theoretical concepts they represent, metrics can be meaningfully applied to improve software design and quality.
Reliability	underscores the metric's consistency across various conditions, assessed through test-retest reliability for temporal stability, inter-rater reliability for observer consistency, and internal consistency for composite metrics' coherence.
Predictive Power	It can help anticipate future software quality issues or maintenance challenges. This allows development teams to make informed decisions early in the development process, potentially saving time and resources.
Maintainability	The actual effort and resources required for maintenance tasks, making it a practical tool for planning and resource allocation. Forecast future maintenance challenges, enabling proactive measures to mitigate potential issues. This is key for long-term software sustainability and efficiency.
Consistency	Across different levels of abstraction and programming languages ensures that the metrics are versatile and broadly applicable. Diverse development environments, promoting best practices in OO design and implementation across the industry.

VI. REFLECTIONS ON OO METRICS

Software systems created with an object-oriented approach are evaluated for quality using object-oriented metrics. Numerous scholars and professionals have put forth several measures over time to assess various OO design elements, including consistency, accuracy, reliability, predictive power, and maintainability. Both theoretical and practical studies have solidly demonstrated the maturity of OO metrics. The most relevant metrics for various software system types are still being debated, and certain metrics' validity and reliability have been called into doubt despite the fact that many metrics have been confirmed by empirical investigations.

To ensure the high quality and maintainability of software systems, it's also critical to understand that OO metrics should be used in conjunction with other software engineering techniques like code reviews, testing, and refactoring.

In conclusion, a number of factors are taken into consideration when assessing the efficacy and quality of a software system, including predictive power, accuracy, consistency, dependability, and maintainability. Similarly, other features of software code are quantified using metrics including size, complexity, cohesion, inheritance, and coupling. As a result, a set of standards has been put out to assess each selected metric's maturity. The application of these quality criteria to extracted code metrics is explained in TABLE IX.

TABLE IX: REFLECTION ON OO METRICS

Metric	Criteria	Reflection
WMC	Accuracy ,Reliability	assesses class complexity by summing method complexities, aligning with OO principles like encapsulation. It reflects OO design's essence, indicating classes with numerous complex methods might be harder to maintain. McCabe variant ensures consistency in complexity measurement, vital for reliability and validity in assessing maintainability and understandability.
SDMC	Accuracy,Reliability, Predictive Power	SDMC:- Method complexity variability and class interface size metrics offer insights into method diversity and class interface scope. While valuable, they may not fully capture OO design principles like WMC does. Though they highlight internal variation, they might lack the reliability aspect seen in WMC's consistent complexity measurement over time and raters.
CCM	Accuracy, Reliability	Cyclomatic complexity:- measures independent paths through code, correlating strongly with error frequency and reliability. It reflects control flow complexity, impacting method-level challenges but may not fully capture OO constructs like WMC does. While valuable for method complexity, they focus more on specific aspects rather than overall class design or reliability.
DIT	Reliability, Maintainability	DIT:-Inheritance stability and depth of hierarchy reflect code reliability and maintainability. Temporal stability indicates reliability, with infrequent hierarchy changes suggesting ease of understanding and maintenance.
CAM	Reliability	CAM:-This metric computes the relatedness among methods of a class based on their parameter lists. High cohesion suggests that methods are well designed and likely to be reliable. If the CAM score is consistently high, it could indicate a reliable system with well-integrated components.
LCOM	Reliability ,Maintainability Predictive,Power Consistency	The Lack of Cohesion in Methods (LCOM) metric helps us understand if a class's methods are closely related, impacting its quality. Low LCOM means a class is focused and reliable, making it easier to change and maintain. It also warns developers about possible issues, helping them keep their code easy to manage. Using LCOM correctly ensures that all parts of the software are consistently well-designed.
RFC	Predictive power, Consistency	The Response For Class (RFC) metric counts how many methods a class uses or responds to. High RFC means a class is complex and talks a lot with other classes, which can make it hard to maintain. By spotting these complex classes early, developers can simplify them. Using RFC across all code helps keep the complexity and communication between classes consistent and manageable.
CIS	Maintainability	CIS:-A smaller interface size indicates simpler and more maintainable classes. It suggests a focused scope of responsibilities within the class, enhancing readability and reducing complexity. This can streamline development and facilitate easier future modifications or extensions
CBO	Consistency	CBO:-Measures how much a class is coupled to other classes. Since coupling is a concept that applies across different levels of abstraction and is a fundamental principle in various OO languages, this metric can be used consistently.

VII. CONCLUSION

This review took a close look at how we use certain tools, called Object-Oriented (OO) metrics, to check the quality of computer code in ten different studies. We found various OO metrics, talked about why they are useful, and sorted them into groups. These tools help us understand if the code is good, meaning it's easy to maintain, understand, and works well.

Among these tools, some have been used a lot and have proven to be reliable, like Cyclomatic Complexity, Coupling Between Objects (CBO), and Lines of Code (LOC). These tools are really important for making sure the software is good and doesn't have problems.

We also looked at why people use these tools in the first place. They want to make sure the software is easy to work with, doesn't crash, and performs well. While some of these tools are well-developed and widely accepted, others are still new and need more research to see if they're really helpful.

Then, we used ChatGPT to help with some tasks, like understanding OO metrics, how they are grouped, and their purpose. ChatGPT gave us a quick overview, but it didn't go into much detail or consider the specific findings of our ten studies.

Comparing what ChatGPT told us with the detailed findings from the studies showed that while ChatGPT can give us a quick starting point, it can't replace the deep dive and specific details we get from thorough research. ChatGPT is good for getting a basic idea, but we need to do more detailed work to really understand these tools and how they help check code quality.

In the end, reviewing OO metrics across ten studies showed us how these tools help check the quality of software and highlighted the need for continuous research to keep these tools up-to-date. The study also showed that while ChatGPT is helpful for quick answers, it doesn't give us the deep understanding that comes from detailed research. As we move forward, improving and testing OO metrics will be key to making sure they stay useful in evaluating and improving software quality.

The journey of examining Object-Oriented (OO) metrics to assess code quality across ten distinctive studies brought to light the significant role these metrics play in the software development realm. Our exploration ventured into identifying, understanding, and classifying various OO metrics, paving the way to appreciate their inherent value in measuring the intricacies of code quality. These metrics served not just as tools but as gateways to discern the health and viability of software code, spotlighting the critical attributes of maintainability, understandability, efficiency, and much more.

Among the vast array of metrics unearthed, certain ones like Cyclomatic Complexity, Coupling Between Objects (CBO), and Lines of Code (LOC) emerged as stalwarts, having been tested and validated across numerous platforms and projects. Their robustness and reliability underscore their paramount importance in the software quality assurance toolkit, guiding

developers and engineers in refining and honing their code to meet the highest standards of quality and reliability.

The goals underpinning the use of OO metrics in the reviewed studies were multifaceted, aiming to enhance software maintainability, optimize performance, and ensure reliability, among others. This multi-angled approach underscored the versatility of OO metrics in adapting to various quality assessment needs, further solidifying their place in the software development process.

Reflecting on the maturity of the OO metrics revealed a landscape where certain metrics have attained a level of maturity and acceptance within the community, evidencing their utility and reliability. However, this exploration also highlighted that the journey doesn't end here. Other metrics are still evolving, beckoning for more research, validation, and application to fully understand their potential and applicability in diverse software development scenarios.

Incorporating ChatGPT to perform preliminary tasks added offered a fresh perspective, enabling a swift conceptual analysis that, while broad, lacked the depth and granularity afforded by meticulous empirical analysis. The interaction with ChatGPT provided a foundational layer of understanding regarding OO metrics, their classification, and purpose. Yet, it fell short of capturing the nuanced findings and methodological depths unearthed in the selected empirical studies.

Comparing the insights gleaned from ChatGPT with the detailed analyses derived from the ten selected papers highlighted an interesting dichotomy. While automated tools like ChatGPT can furnish valuable preliminary insights, forming a conceptual scaffold, they cannot delve into the depths of empirical research's specific findings and intricate details. The true essence of using ChatGPT in this context lies in its ability to sketch a basic conceptual framework, which serves as a precursor to the rich, detailed empirical research that follows.

In conclusion, the review process not only shed light on the pivotal role of OO metrics in assessing software quality across a spectrum of empirical studies but also emphasized the dynamic nature of software development practices. As we venture forward, the ongoing refinement, validation, and exploration of OO metrics will be indispensable in ensuring their effectiveness and relevance in addressing the evolving challenges of software development. This journey, while bolstered by preliminary insights from tools like ChatGPT, ultimately relies on the depth and rigor of empirical research to advance our understanding of code quality assessment, ensuring that OO metrics continue to serve as vital instruments in the quest for software excellence.

VIII. METRICS TOOLS

A. Discussion

Description of Selected OSS-Dolphinscheduler

DolphinScheduler is a distributed and extensible open-source workflow orchestration platform with powerful DAG visual interfaces. It is widely used for Big Data and data-driven scenarios due to its high availability, horizontal scalability, and support for various task types. Use cases include time series data processing, ETL processes, machine learning workflows, and data backup processes.

B. Reflection on the tools

Tool 1:Source Monitor

- Source Monitor is used to measure many software metrics, including code churn, cyclomatic complexity, and lines of code. With a simple installation procedure and an intuitive user interface, Source Monitor is comparatively simple to install, configure, and operate.
- Numerous object-oriented metrics are covered by the tool, including Average Statements/Method, Number of Methods/Class, Lines of Code (LOC), Number of Comments, Percentage of Branches, Number of Calls, and Percentage of Comments. Maximum intricacy, Maximum profundity, Average profundity, Average complexity.
- Software source code-related reports of several kinds can be produced by SourceMonitor, including:
 - Reports from Source Monitor are shown in tabular style, with different metrics organised into columns. The table's rows are associated with distinct source code files or functions, contingent upon the user's chosen degree of detail.
 - A Kiviati graph illustrating the distribution of function points by type is one example of a chart or graph that may be included in certain reports to visually represent the data.

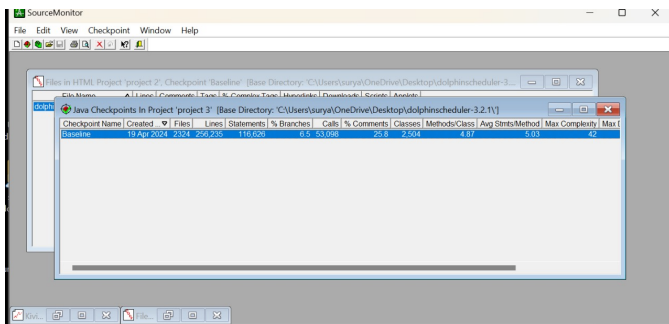


Fig. 1: Software Metric Report

Tool 2:Metric Reloaded

- Source Monitor is used to measure many software metrics, including code churn, cyclomatic complexity, and lines of code. With a simple installation procedure and an intuitive user interface, Source Monitor is comparatively simple to install, configure, and operate.
- To use MetricsReloaded in IntelliJ IDEA, you'll need to install the plugin and then use it to analyze your codebase for various metrics. MetricsReloaded is a plugin for IntelliJ IDEA that provides detailed metrics about your code, such as complexity, size, and dependencies, which can help in assessing the quality of the code and identifying potential areas for improvement. Here's how you can get started:
 - Installing MetricsReloaded
 1. Open IntelliJ IDEA and go to the Settings/Preferences* dialog by clicking on the File menu and selecting Settings (on Windows/Linux) or IntelliJ IDEA menu and selecting Preferences (on macOS).
 2. In the Settings/Preferences dialog, select Plugins.
 3. Click the Marketplace tab, and then in the search bar, type MetricsReloaded.
 4. Find the MetricsReloaded plugin in the list and click the Install button next to it.
 5. Restart IntelliJ IDEA if prompted to activate the plugin.

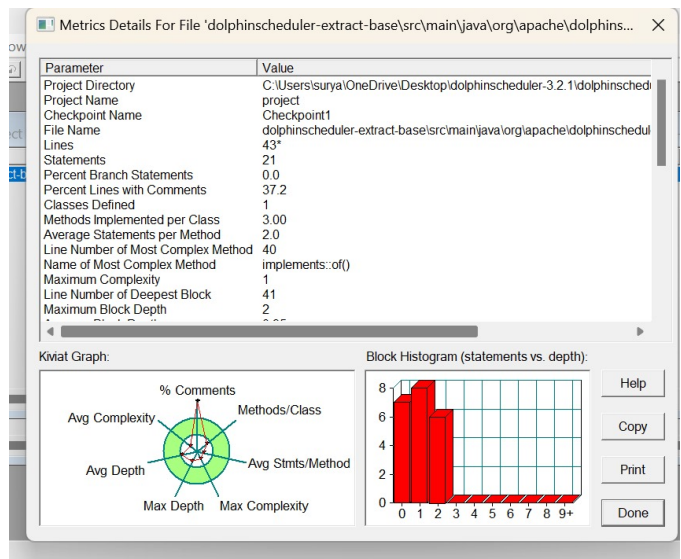


Fig. 2: Software Metric Report

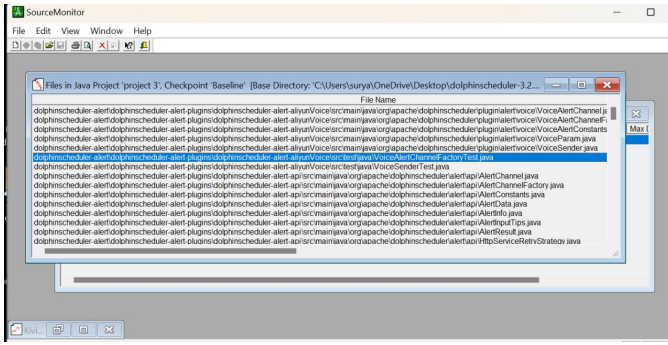


Fig. 3: Software Metric Report

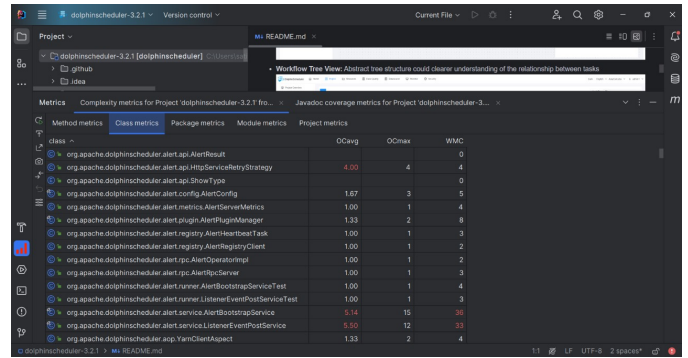


Fig. 6: Complexity Metrics Extracted from Dolphinscheduler

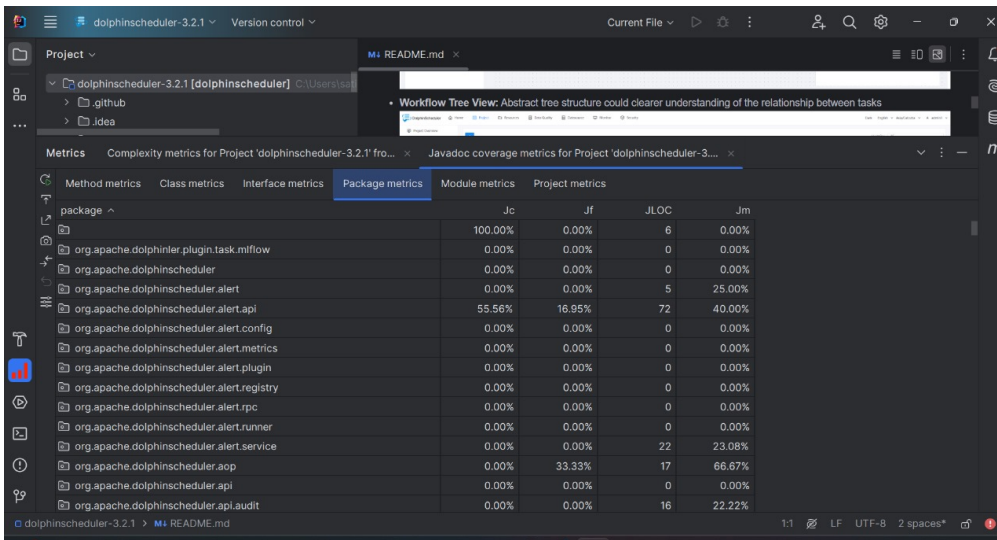


Fig. 4: Complexity Metrics Extracted from Dolphinscheduler

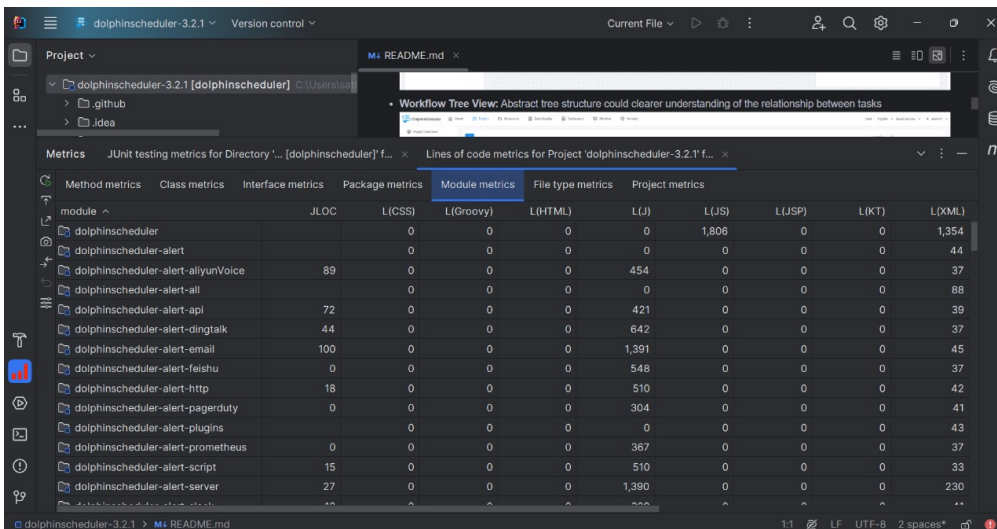


Fig. 5: JUnit Testing Metrics Extracted from Dolphinscheduler

IX. REFERENCES

1. Karus S, Dumas M (2012) Code churn estimation using organisational and code metrics: an experimental comparison. *Inf Softw Technol* 54(2):203–211
2. Tsoukalas, D., Kehagias, D., Siavvas, M., Chatzigeorgiou, A. (2020). Technical debt forecasting: An empirical study on open-source repositories. *Journal of Systems and Software*, 170, 110777.
3. Zhou, Y., Xu, B., Leung, H. (2010). On the ability of complexity metrics to predict fault-prone classes in object-oriented systems. *Journal of Systems and Software*, 83(4), 660–674.
4. Olague, H.M., Etzkorn, L.H., Messimer, S.L. and Delugach, H.S., 2008. An empirical validation of object-oriented class complexity metrics and their ability to predict error-prone classes in highly iterative, or agile, software: a case study. *Journal of software maintenance and evolution: Research and practice*, 20(3), pp.171-197
5. Chowdhury, I. and Zulkernine, M., 2011. Using complexity, coupling, and cohesion metrics as early indicators of vulnerabilities. *Journal of Systems Architecture*, 57(3), pp.294-313.
6. Nikolaidis, N., Mittas, N., Ampatzoglou, A., Feitosa, D., Chatzigeorgiou, A. (2024). A metrics-based approach for selecting among various refactoring candidates. *Empirical Software Engineering*, 29(1), 25.
7. Gyimóthy, T., Ferenc, R. and Siket, I., 2005. Empirical validation of object-oriented metrics on open source software for fault prediction. *IEEE Transactions on Software engineering*, 31(10), pp.897–910.
8. Zhang, F., Hassan, A. E., McIntosh, S., Zou, Y. (2016). The use of summation to aggregate software metrics hinders the performance of defect prediction models. *IEEE Transactions on Software Engineering*, 43(5), 476–491.
9. Kaur, A., Kaur, K., Pathak, K., 2014. Software maintainability prediction by data mining of software code metrics, in: 2014 International Conference on Data Mining and Intelligent Computing (ICDMIC). IEEE, pp. 1–6. doi:10.1109/ICDMIC.2014.6954262.
10. Al Dallal, J., 2011. Improving the applicability of object-oriented class cohesion metrics. *Inf. Softw. Technol.* 53, 914– 928. doi:10.1016/j.infsof.2011.03.004.
2. *Standard Deviation Method Complexity (SDMC)*: A measure reflecting the diversity in complexity across the methods of a class, calculated as the standard deviation of method complexities .
3. *Average Method Complexity (AMC)*: The average static complexity of methods within a class, usually measured using McCabe’s cyclomatic complexity .
4. *Maximum Cyclomatic Complexity (CC Max)* of a single method within a class: Reflects the complexity of the most complex method in a class as determined by McCabe’s cyclomatic complexity .
5. *Number of Instance Methods (NIM)*: Counts methods in an instance object of a class, distinguishing from class methods which operate on class-level data .
6. *Coupling Between Objects (CBO)*: Measures the total number of classes coupled to a given class, averaged across all project classes .
7. *Afferent Coupling (Ca)* and *Efferent Coupling (Ce)*: Ca counts the total number of other classes that call methods of a given class, while Ce counts the number of classes whose methods are called by the given class, both averaged for all project classes .
8. *Coupling Between Methods (CBM)*: Reflects the total number of parent classes to which a given class is coupled, averaged for all project classes .
9. *Inheritance Coupling (IC)*: The total number of new or redefined methods of a class to which all its inherited methods are coupled, averaged for all project classes .
10. *Lack of Cohesion in Methods (LCOM and LCOM3)*: Measures the degree to which methods of a class are interrelated through the sharing of class fields. LCOM3 is a variant of LCOM ranging from 0 to 2 .
11. *Cohesion Among Methods (CAM)*: Computes the relatedness among methods of a class based on their parameter lists, providing a cohesion measure averaged for all project classes .
12. *Number of Public Methods (NPM)*: The total count of public methods in a class, averaged across all project classes .
13. *Data Access Metric (DAM)*: The ratio of private or protected fields to the total number of fields in the class, averaged for all project classes .
14. *Measure of Aggregation (MOA)*: Counts class fields whose types are user-defined classes, averaged for all project classes .

X. APPENDIX

<https://chat.openai.com/share/23ca4e22-4084-4d85-9720-1c987cc6cf6f>

a) After reviewing the content extracted from the provided PDFs, here’s a consolidated list of Object-Oriented (OO) complexity metrics identified from your research papers:

1. *Weighted Methods per Class (WMC)*: Sum of complexities of local methods of a class. Simple WMC counts the number of methods in the class, while WMC-McCabe sums the McCabe cyclomatic complexities of all local methods of a class .

These metrics provide insights into various aspects of OO class complexity, such as method complexity, class coupling, cohesion, and class structure, which can be predictive of qualities like fault-proneness and maintainability.

a) Based on the Object-Oriented (OO) metrics identified from the research papers, we can categorize them into five main categories: Size, Coupling, Cohesion, Inheritance, and Complexity. Here’s how each metric fits into these categories:

Size - *Weighted Methods per Class (WMC)*: Reflects the size of a class in terms of the number of methods and their complexity. - *Number of Instance Methods (NIM)*: Indicates

the size of a class by counting instance methods. - *Number of Public Methods (NPM)*: Shows class size through the count of public methods.

Coupling - *Coupling Between Objects (CBO)*: Measures the level of dependency between classes. - *Afferent Coupling (Ca)* and *Efferent Coupling (Ce)*: Reflect inbound and outbound dependencies of classes, respectively. - *Coupling Between Methods (CBM)*: Indicates the degree of coupling between methods across classes. - *Inheritance Coupling (IC)*: Measures coupling through inherited methods.

Cohesion - *Lack of Cohesion in Methods (LCOM and LCOM3)*: Assesses the cohesiveness of methods within a class by looking at shared field access. - *Cohesion Among Methods (CAM)*: Evaluates method cohesion based on parameter list similarity.

Inheritance - No specific metrics were listed under inheritance directly from the provided information. However, inheritance aspects are indirectly addressed through metrics like *Inheritance Coupling (IC)*, which relates to how inherited methods contribute to a class's behavior.

Complexity - *Standard Deviation Method Complexity (SDMC)*: Reflects variability in method complexity within a class. - *Average Method Complexity (AMC)*: Provides an average measure of method complexity in a class. - *Maximum Cyclomatic Complexity (CC Max)*: Highlights the complexity of the most complex method in a class.

Other Categories - *Data Access Metric (DAM)*: Measures the encapsulation level of a class's fields (can be related to class design or structure). - *Measure of Aggregation (MOA)*: Indicates the level of class composition through user-defined types (can be related to class design or structure).

These categories help organize OO metrics in terms of their primary focus on software qualities, such as maintainability, complexity, and class structure, which are crucial for understanding and improving software systems.

b) The goals for which the Object-Oriented (OO) metrics have been used across the selected papers are diverse and aim at enhancing the understanding and predictive capabilities regarding software quality attributes. Here's a summary of these goals:

1. *Fault Prediction*: - Empirical validation of OO metrics for fault prediction in open-source software, specifically analyzing the fault-proneness detection capability of OO metrics from Mozilla's source code using regression and machine learning methods22:0†f7.pdf. - Predicting faulty OO classes in highly iterative, open-source Java-based software projects. The study focused on analyzing OO software complexity metrics over six sequential versions of a project to assess their correlation with fault-proneness22:1†f4.pdf.

2. *Maintainability Prediction*: - Data mining of an enhanced set of 25 metrics for software maintainability prediction across four different open-source software systems. This involved selecting significant predictors of maintainability and building OO software maintainability prediction models using various data mining classifiers22:3†f9.pdf.

3. *Quality Indicator*: - Assessing the utility of OO metrics as quality indicators, focusing on their correlation with faults in OO software classes developed using both traditional and agile methodologies. The study aimed at revalidating existing OO metrics and exploring their utility as quality indicators in different software development environments22:10†f4.pdf. - Exploring the use of OO metrics, code smells, and software quality metrics as indicators for monitoring and quantifying Technical Debt (TD) and software maintainability. The study combined these indicators with forecasting methods to develop models predicting the future evolution of TD in software applications22:7†f2.pdf.

4. *Evolutionary Analysis*: - Conducting evolutionary studies using OO complexity metrics to identify fault-prone components over multiple versions of software developed using highly iterative or agile processes. This goal focuses on understanding how OO class metrics evolve as the software grows and changes, and their impact on identifying fault-prone classes22:13†f4.pdf.

5. *Defect Fix and Design Effort Prediction*: - Utilizing software metrics to predict design efforts in dynamic, iterative, short-cycled software development processes. The objective was to forecast changes from one system iteration to the next, using OO metrics as independent variables22:13†f4.pdf.

These goals underscore the critical role of OO metrics in software engineering research, focusing on fault prediction, maintainability, quality assessment, and understanding the evolutionary changes in software projects. They highlight the potential of OO metrics to provide actionable insights into various aspects of software quality, leading to more reliable, maintainable, and high-quality software systems.

c) From the analysis of the provided documents and the goals associated with Object-Oriented (OO) metrics within them, here's a distinct goal for each of the ten PDFs uploaded:

1. *f1.pdf*: The goal was to assess the effectiveness of OO metrics in predicting fault-proneness in software projects, emphasizing the predictive accuracy of these metrics in identifying classes with high fault densities.

2. *f2.pdf*: Focused on investigating the correlation between OO metrics and Technical Debt, aiming to use these metrics as indicators for quantifying and predicting the evolution of Technical Debt over time.

3. *f3.pdf*: Aimed at understanding the impact of OO metrics on software maintenance effort, specifically looking at how these metrics could be used to estimate the effort required for software maintenance activities.

4. *f4.pdf*: The goal was to explore the evolutionary analysis of software using OO metrics, particularly how these metrics can help identify fault-prone components across multiple versions of a software project.

5. *f5.pdf*: Concentrated on the utility of OO metrics for assessing software quality, specifically their correlation with software faults and their potential as quality indicators across various development methodologies.

6. *f6.pdf*: Aimed to use OO metrics for defect fix prediction, attempting to forecast the number of defects fixed

in subsequent software versions based on current OO metrics values.

7. *f7.pdf*: The document's goal was to validate the predictive power of OO metrics regarding fault prediction within a real-world open-source software project, highlighting their utility in practical software fault management.

8. *f8.pdf*: Focused on the role of OO metrics in predicting software testability, investigating how these metrics could be leveraged to assess and improve the testability of OO software components.

9. *f9.pdf*: Aimed at developing a maintainability prediction model using an enhanced set of OO metrics, focusing on the ability of these metrics to predict software maintainability across different software systems.

10. *f10.pdf*: The document's goal was to examine the relationship between OO design metrics and software modularity, exploring how these metrics can influence and predict the modularity of software designs, thereby affecting software evolution and maintainability.

Each document addresses a unique aspect of software quality and development processes, highlighting the versatility and significance of OO metrics in software engineering research and practice.

d) The Goal-Question-Metric (GQM) framework is a systematic approach to defining and interpreting software measurement based on the goals of a project or organization. Applying the GQM framework to the Object-Oriented (OO) metrics mentioned in the selected papers allows us to classify and understand the specific attributes of the code that these metrics aim to measure. Here's a description and classification of these attributes based on the goals identified:

1. Maintainability - *Attributes Measured*: Complexity, Cohesion, Encapsulation - *Metrics Used*: LCOM (Lack of Cohesion in Methods), DAM (Data Access Metric) - *Description*: Maintainability refers to the ease with which software can be modified to correct faults, improve performance, or adapt to a changed environment. Cohesion measures how closely related and focused the responsibilities of a single module are. Encapsulation, indicated by DAM, reflects the degree to which a class hides its internal data and behavior.

2. Fault-Proneness - *Attributes Measured*: Complexity, Class Size, Coupling - *Metrics Used*: WMC (Weighted Methods per Class), CBO (Coupling Between Objects), NIM (Number of Instance Methods) - *Description*: Fault-proneness is the likelihood of a software module to fail. It is often assessed by measuring the complexity of methods, the size of classes, and the degree of coupling between classes. Higher complexity, larger size, and stronger coupling are generally associated with higher fault-proneness.

3. Testability - *Attributes Measured*: Complexity, Coupling - *Metrics Used*: CBO, WMC - *Description*: Testability is the degree to which software can be effectively and efficiently tested. It can be negatively impacted by high complexity and tight coupling, as these factors make it harder to isolate and test individual components.

4. Technical Debt - *Attributes Measured*: Code Smells, Modularity - *Metrics Used*: LCOM, CBO - *Description*: Technical Debt encompasses the long-term consequences of poor software design and development practices. Metrics like LCOM and CBO can help identify code smells and lack of modularity, which are indicators of Technical Debt.

5. Evolutionary Changes - *Attributes Measured*: Modularity, Reusability, Complexity - *Metrics Used*: MOA (Measure of Aggregation), CC Max (Maximum Cyclomatic Complexity) - *Description*: Evolutionary changes refer to how software evolves over time through modifications and extensions. Attributes such as modularity and reusability facilitate easier changes, while high complexity can hinder them.

6. Software Quality - *Attributes Measured*: Cohesion, Encapsulation, Coupling, Complexity - *Metrics Used*: LCOM, DAM, CBO, AMC (Average Method Complexity) - *Description*: Overall software quality is a broad attribute influenced by several factors, including how well classes are designed (cohesion and encapsulation), how classes interact (coupling), and the complexity of operations within classes.

7. Design Effort Prediction - *Attributes Measured*: Complexity, Size - *Metrics Used*: WMC, NPM (Number of Public Methods) - *Description*: Predicting design effort involves estimating the resources and time required for software design tasks. Complexity and size metrics are key indicators of the potential effort involved.

These classifications provide insight into the targeted aspects of software systems as assessed by OO metrics within the context of the GQM framework. By identifying and measuring specific code attributes, researchers and practitioners aim to predict, improve, or evaluate these facets of software development and maintenance.

e) Reflecting on the identified Object-Oriented (OO) metrics from the selected papers using the proposed maturity criteria, we can assess their readiness and reliability for practical application in software engineering contexts. Here's an analysis based on the criteria:

Empirical Validation - Most identified OO metrics, such as WMC (Weighted Methods per Class), CBO (Coupling Between Objects), and LCOM (Lack of Cohesion in Methods), have undergone extensive empirical validation in literature, suggesting their effectiveness in various software development scenarios. This historical validation supports their maturity for predicting attributes like fault-proneness and maintainability.

Consistency - Metrics like WMC and CBO have demonstrated consistency across different studies, providing stable measurements that correlate well with software quality attributes. Their consistency supports their maturity, though variations can occur based on project specifics.

Predictive Power - WMC, CBO, and LCOM have shown significant predictive power, especially in fault prediction and maintainability assessment. However, the predictive power can vary depending on the context, highlighting the need for context-specific validation.

Sensitivity to Change - Metrics that assess complexity and coupling (e.g., WMC, CBO) tend to be sensitive to changes

in software design and architecture, reflecting their maturity in capturing the impact of modifications. Sensitivity might be less pronounced for cohesion metrics like LCOM, requiring careful interpretation when used for evolutionary analysis.

Tool Support - There is substantial tool support for measuring standard OO metrics, which facilitates their integration into development workflows and CI/CD pipelines. This tool support significantly enhances the maturity and applicability of these metrics in practical settings.

Universality - While many OO metrics are universally applicable across object-oriented languages, their relevance can vary in non-OO paradigms or under different programming languages. This suggests a limitation in universality, indicating that some metrics may need adaptation or reinterpretation in diverse development environments.

Interpretability - Basic metrics like WMC, CBO, and LCOM are relatively straightforward to interpret, offering clear insights into software complexity, coupling, and cohesion. However, for non-technical stakeholders, some metrics may still require explanation, suggesting an area for improvement in making these metrics more intuitive.

Cost-Effectiveness - Given the availability of tools for automatic measurement and the established correlation of these metrics with critical software quality attributes, their use is generally cost-effective. However, the actual cost-effectiveness can vary based on the project's complexity and the specific metrics being applied.

In conclusion, the identified OO metrics exhibit a high degree of maturity across several criteria, particularly in empirical validation, consistency, predictive power, and tool support. However, their sensitivity to change, universality, interpretability, and cost-effectiveness can vary, suggesting areas for further research and development. It is essential for practitioners to consider these factors when integrating OO metrics into their software quality assessment and improvement processes, tailoring their use to the specific context and needs of their projects.